

SISTEMA DE CONTROL DE SALDO

Análisis, algoritmo, pseudocódigo, flujograma, prueba de escritorio y codificación

ENUNCIADO DEL EJERCICIO

Implemente un sistema con saldo inicial y las opciones: consultar saldo, depositar, retirar, ver número de transacciones y salir. No permita valores negativos ni retiros superiores al saldo disponible.

Estructura sugerida: do-while. Conceptos: menú, acumuladores, contador y validaciones.

1. ANÁLISIS DE DATOS

1.1 Entrada de datos

1. saldo: valor real inicial ingresado por el usuario, mayor o igual que cero. Se vuelve a solicitar si es negativo.
2. opcion: entero de 1 a 5: consultar saldo, depositar, retirar, consultar transacciones y salir, respectivamente.
3. monto: valor real solicitado al depositar o retirar. Debe ser mayor que cero; para retirar, también debe ser menor o igual que saldo.
4. Se utilizan entradas numéricas, sin letras. Los valores decimales se ingresan con el separador que acepte el entorno.

1.2 Proceso

5. Inicializar transacciones en 0. Solicitar el saldo inicial con do-while hasta que sea no negativo.
6. Mostrar el menú con do-while y leer opcion. Evaluarla mediante switch.
7. Opción 1: mostrar saldo. Opción 2: leer monto; si es positivo, sumar al saldo e incrementar transacciones. Si no lo es, mostrar un aviso.
8. Opción 3: leer monto; rechazarlo si es menor o igual que cero o supera el saldo. Si es válido, restarlo del saldo e incrementar transacciones.
9. Opción 4: mostrar transacciones. Opción 5: mostrar la despedida. Para otra opción, mostrar un aviso.
10. Volver al menú mientras opcion sea diferente de 5. Un movimiento rechazado no cambia saldo ni transacciones.

1.3 Salida de datos

11. Saldo actual, confirmaciones de depósitos y retiros, y número de transacciones realizadas.
12. Avisos de saldo inicial negativo, monto no positivo, saldo insuficiente u opción inválida.
13. Mensaje de finalización al seleccionar 5.

2. ALGORITMO

Criterios del programa: el saldo inicial no cuenta como transacción. Solo se cuentan depósitos y retiros exitosos; las consultas no se cuentan. Los montos iguales a cero se rechazan porque no representan un movimiento de dinero.

14. Inicio.

15. Declarar saldo y monto como reales, y opcion y transacciones como enteros. Inicializar el contador de transacciones en cero.
16. Solicitar y leer el saldo inicial. Si es negativo, mostrar un aviso y repetir la solicitud hasta recibir un valor mayor o igual que cero.
17. Mostrar las cinco opciones y leer la opción elegida.
18. Si elige consultar saldo, mostrar el saldo actual.
19. Si elige depositar, solicitar el monto. Si es menor o igual que cero, mostrar el aviso. En caso contrario, sumarlo al saldo, aumentar el contador en uno y mostrar la confirmación con el nuevo saldo.
20. Si elige retirar, solicitar el monto. Si es menor o igual que cero, mostrar el aviso. Si supera el saldo, informar que el saldo es insuficiente. De lo contrario, restarlo del saldo, aumentar el contador en uno y mostrar la confirmación.
21. Si elige ver el número de transacciones, mostrar el contador.
22. Si elige salir, mostrar la despedida. Ante otra opción, mostrar el aviso de opción inválida.
23. Comprobar la condición del ciclo: si la opción es diferente de cinco, volver al menú; si es cinco, terminar.

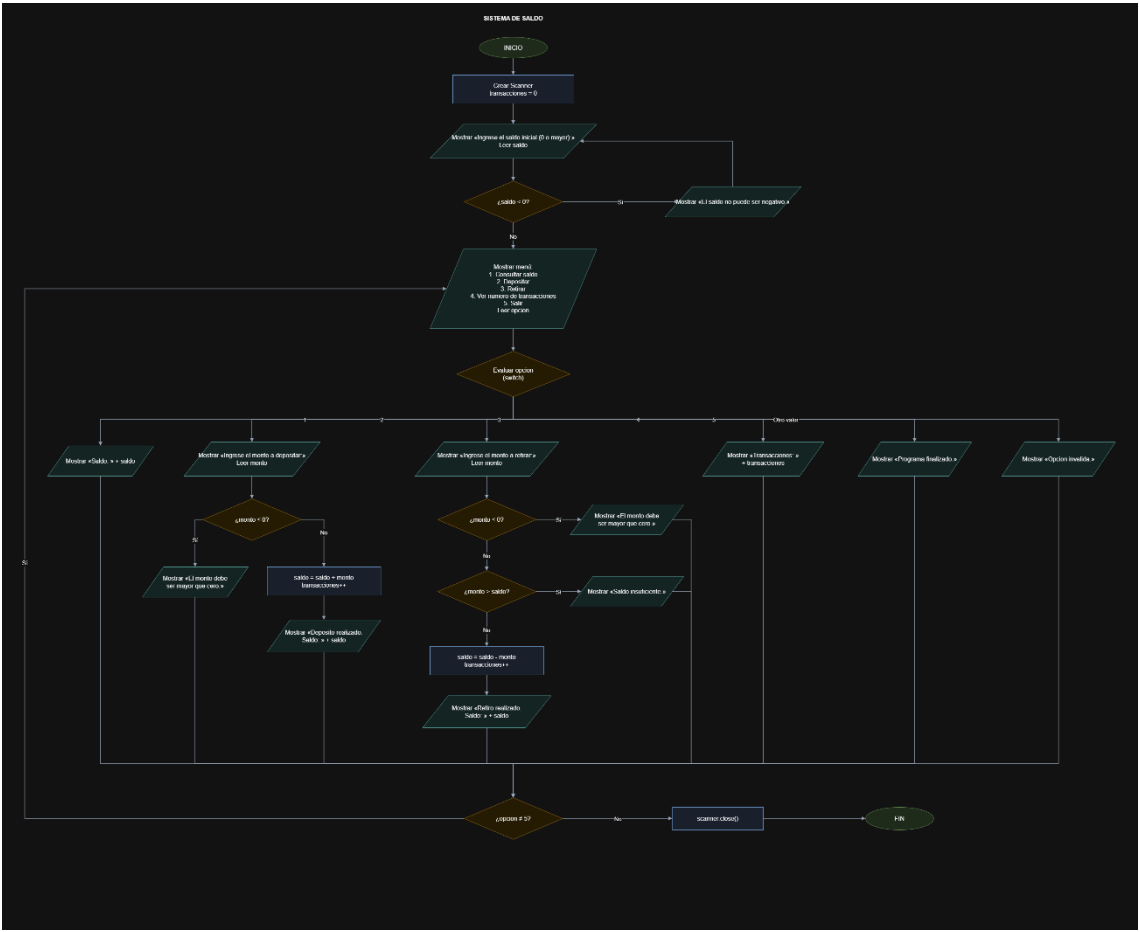
24. Fin.

3. PSEUDOCÓDIGO

```
Algoritmo SistemaSaldo
  Definir saldo, monto Como Real;
  Definir opcion, transacciones Como Entero;
  transacciones <- 0;
  Repetir
    Escribir "Ingrese el saldo inicial (0 o mayor):";
    Leer saldo;
    Si saldo < 0 Entonces
      Escribir "El saldo no puede ser negativo.";
    FinSi
  Hasta Que saldo >= 0
  Repetir
    Escribir "1. Consultar saldo";
    Escribir "2. Depositar";
    Escribir "3. Retirar";
    Escribir "4. Ver numero de transacciones";
    Escribir "5. Salir";
    Leer opcion;
    Segun opcion Hacer
      1:
        Escribir "Saldo: ", saldo;
      2:
        Escribir "Ingrese el monto a depositar:";
        Leer monto;
        Si monto <= 0 Entonces
          Escribir "El monto debe ser mayor que cero.";
        SiNo
          saldo <- saldo + monto;
          transacciones <- transacciones + 1;
          Escribir "Deposito realizado. Saldo: ", saldo;
        FinSi
      3:
        Escribir "Ingrese el monto a retirar:";
        Leer monto;
        Si monto <= 0 Entonces
          Escribir "El monto debe ser mayor que cero.";
        SiNo
          Si monto > saldo Entonces
            Escribir "Saldo insuficiente.";
          SiNo
            saldo <- saldo - monto;
            transacciones <- transacciones + 1;
            Escribir "Retiro realizado. Saldo: ", saldo;
          FinSi
        FinSi
      4:
        Escribir "Transacciones: ", transacciones;
      5:
        Escribir "Programa finalizado.";
    De Otro Modo:
      Escribir "Opcion invalida.";
    FinSegun
  Hasta Que opcion = 5
FinAlgoritmo
```

4. DIAGRAMA DE FLUJO

Menú y validación del saldo inicial



5. PRUEBA DE ESCRITORIO

Caso 1 — Saldo cero y consultas

Variable	Valor
saldo inicial	0
opcion en orden	1, 4, 5
saldo final	0
transacciones	0

Operaciones:

saldo = 0 cumple saldo $\geq$ 0. Se acepta en el primer intento.

Opción 1: muestra saldo = 0. Opción 4: muestra transacciones = 0.

Las consultas no modifican ninguna variable. La opción 5 finaliza el programa.

Caso 2 — Depósito y retiro correctos

Variable	Valor
saldo inicial	200
opcion en orden	2, 3, 1, 4, 5
monto en orden	50 para depositar; 80 para retirar
saldo en orden	200 $\rightarrow$ 250 $\rightarrow$ 170
transacciones en orden	0 $\rightarrow$ 1 $\rightarrow$ 2

Operaciones:

Depósito: $50 > 0$. saldo = $200 + 50 = 250$; transacciones = $0 + 1 = 1$.

Retiro: $80 > 0$ y $80 \leq 250$. saldo = $250 - 80 = 170$; transacciones = $1 + 1 = 2$.

Consultar saldo muestra 170. Consultar transacciones muestra 2. Ambas consultas mantienen los valores y la opción 5 termina el programa.

Caso 3 — Validaciones y retiro total

Variable	Valor
saldo inicial ingresado	-20 (rechazado), 50 (aceptado)
opcion en orden	6, 2, 2, 3, 3, 3, 1, 4, 5
montos de depósito	-10; 0 (ambos rechazados)
montos de retiro	-5; 60 (rechazados); 50 (aceptado)
saldo final / transacciones	0 / 1

Operaciones:

Saldo inicial: $-20 < 0$; se muestra el aviso y se solicita otra entrada. Se acepta 50 porque no es negativo.

Opción 6: se muestra «Opcion invalida.». saldo = 50 y transacciones = 0.

Depósitos de -10 y 0: ambos cumplen monto ≤ 0 . Se rechazan y se mantiene saldo = 50; transacciones = 0.

Retiro de -5: se rechaza por monto ≤ 0 . Retiro de 60: se rechaza porque $60 > 50$; aparece «Saldo insuficiente.».

Retiro de 50: es positivo y no supera el saldo. saldo = $50 - 50 = 0$; transacciones = $0 + 1 = 1$.

Las opciones 1 y 4 muestran 0 y 1, respectivamente. La opción 5 finaliza.

Resumen de los casos

Caso	Saldo inicial válido	Depósitos válidos	Retiros válidos	Saldo final	Total de trans.
1	0	0	0	0	0
2	200	1	1	170	2
3	50	0	1	0	1

Comprobación de ejecución:

Se compilaron Java y C++ y se ejecutaron los tres casos en ambos lenguajes. Se verificaron saldos, contadores y avisos. Los movimientos rechazados no alteran el saldo ni el contador; retirar exactamente el saldo disponible deja el saldo en cero.

6. CÓDIGO

JAVA

```
import java.util.Scanner;

public class SistemaSaldo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        double saldo, monto;
        int opcion, transacciones = 0;

        do {
            System.out.println("Ingrese el saldo inicial (0 o mayor):");
            saldo = scanner.nextDouble();
            if (saldo < 0) {
                System.out.println("El saldo no puede ser negativo.");
            }
        } while (saldo < 0);

        do {
            System.out.println("1. Consultar saldo");
            System.out.println("2. Depositar");
            System.out.println("3. Retirar");
            System.out.println("4. Ver numero de transacciones");
            System.out.println("5. Salir");
            opcion = scanner.nextInt();
            switch (opcion) {
                case 1:
                    System.out.println("Saldo: " + saldo);
                    break;
                case 2:
                    System.out.println("Ingrese el monto a depositar:");
                    monto = scanner.nextDouble();
                    if (monto <= 0) {
                        System.out.println("El monto debe ser mayor que cero.");
                    } else {
                        saldo = saldo + monto;
                        transacciones++;
                        System.out.println("Deposito realizado. Saldo: " + saldo);
                    }
                    break;
                case 3:
                    System.out.println("Ingrese el monto a retirar:");
                    monto = scanner.nextDouble();
                    if (monto <= 0) {
                        System.out.println("El monto debe ser mayor que cero.");
                    } else if (monto > saldo) {
                        System.out.println("Saldo insuficiente.");
                    } else {
                        saldo = saldo - monto;
                        transacciones++;
                        System.out.println("Retiro realizado. Saldo: " + saldo);
                    }
                    break;
                case 4:
                    System.out.println("Transacciones: " + transacciones);
                    break;
                case 5:
                    System.out.println("Programa finalizado.");
                    break;
                default:
                    System.out.println("Opcion invalida.");
                    break;
            }
        } while (opcion != 5);
        scanner.close();
    }
}
```

C++

```
#include <iostream>
using namespace std;

int main() {
    double saldo, monto;
    int opcion, transacciones = 0;

    do {
        cout << "Ingrese el saldo inicial (0 o mayor):" << endl;
        cin >> saldo;
        if (saldo < 0) {
            cout << "El saldo no puede ser negativo." << endl;
        }
    } while (saldo < 0);

    do {
        cout << "1. Consultar saldo" << endl;
        cout << "2. Depositar" << endl;
        cout << "3. Retirar" << endl;
        cout << "4. Ver numero de transacciones" << endl;
        cout << "5. Salir" << endl;
        cin >> opcion;
        switch (opcion) {
            case 1:
                cout << "Saldo: " << saldo << endl;
                break;
            case 2:
                cout << "Ingrese el monto a depositar:" << endl;
                cin >> monto;
                if (monto <= 0) {
                    cout << "El monto debe ser mayor que cero." << endl;
                } else {
                    saldo = saldo + monto;
                    transacciones++;
                    cout << "Deposito realizado. Saldo: " << saldo << endl;
                }
                break;
            case 3:
                cout << "Ingrese el monto a retirar:" << endl;
                cin >> monto;
                if (monto <= 0) {
                    cout << "El monto debe ser mayor que cero." << endl;
                } else if (monto > saldo) {
                    cout << "Saldo insuficiente." << endl;
                } else {
                    saldo = saldo - monto;
                    transacciones++;
                    cout << "Retiro realizado. Saldo: " << saldo << endl;
                }
                break;
            case 4:
                cout << "Transacciones: " << transacciones << endl;
                break;
            case 5:
                cout << "Programa finalizado." << endl;
                break;
            default:
                cout << "Opcion invalida." << endl;
                break;
        }
    } while (opcion != 5);
    return 0;
}
```